The bridge between Cody and Codex

CCP discovers a device, saves a trusted pairing, and checks that its paired PC is still there.

Cody's Codex Plugin

A small Python service on your Windows PC. Codex calls its local tools; Cody talks to it over the LAN.

Cody finds CCP, not arbitrary Codex windows or chats.



01 / Discover

Find pairing-ready Cody devices without typing an IP address.

02 / Pair

Use the display code to retain the intended PC/device relationship.

03 / Stay in touch

Authenticate repeated replies from the retained PC identity.

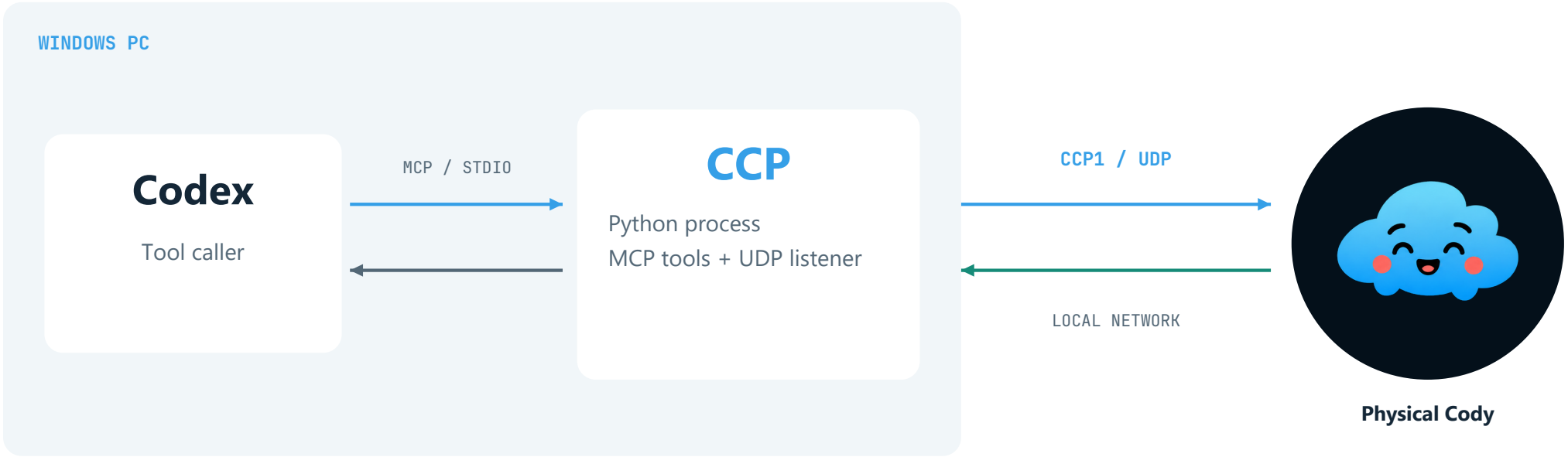
SOURCE / CCP1.md; README.md

NOTES / QUESTIONS

Copyright 2026 / confidential

Two links. One local bridge.

The Codex-to-plugin connection is separate from the plugin-to-device network protocol.



Tools exposed to Codex

ccp_discover_devices
ccp_pair_device
ccp_connection_status

Not implemented by this link

No Codex chat selection, task stream or project/message transport. No raw Codex service is exposed on the LAN.

SOURCE / mcp.json; ccp_server.py: CcpService, CompanionResponder

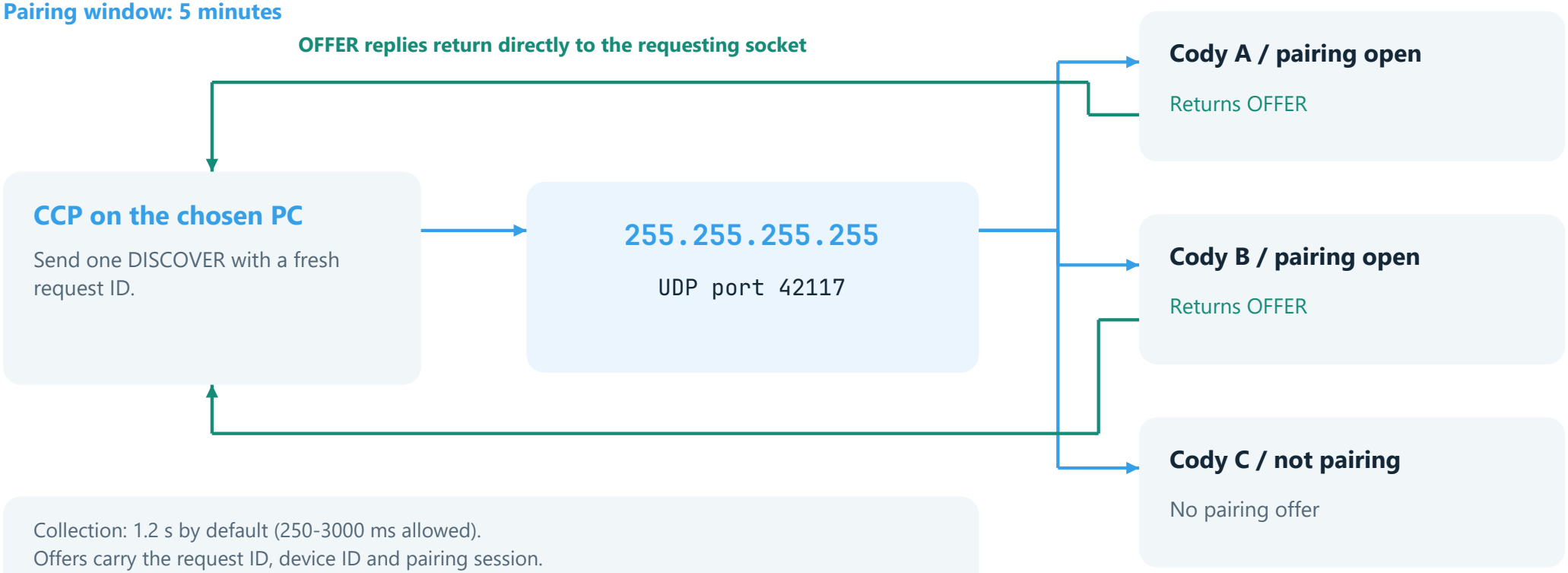
NOTES / QUESTIONS

Copyright 2026 / confidential

First discovery: the PC asks the LAN

Only devices with an open pairing window offer themselves for pairing.

Pairing window: 5 minutes



Collection: 1.2 s by default (250-3000 ms allowed).
Offers carry the request ID, device ID and pairing session.
The IP address is observed from the reply, not entered by you.

SOURCE / ccp_server.py: discover; LUNASHELL_codex_link.cpp: sendOffer

NOTES / QUESTIONS

Copyright 2026 / confidential

The six characters select the right Cody

An offer is a candidate, not permission to pair. CCP must verify it using the user's code.

DISPLAY CODE

K7M4Q2

→ **Code + each offer's salt**

PBKDF2-HMAC-SHA256 / 50,000 iterations

EXAMPLE CODE

Offer A

Recompute its HMAC tag using the derived verifier.

NO MATCH

Offer B

Recompute its HMAC tag using the derived verifier.

MATCH

Offer C

Recompute its HMAC tag using the derived verifier.

NO MATCH

The code is not sent in clear text in CCP1 LAN datagrams.

No match

Stop. Check the code or refresh the offer.

Exactly one match

Continue with this device and session.

More than one match

Stop. Restart pairing; never pick the first.

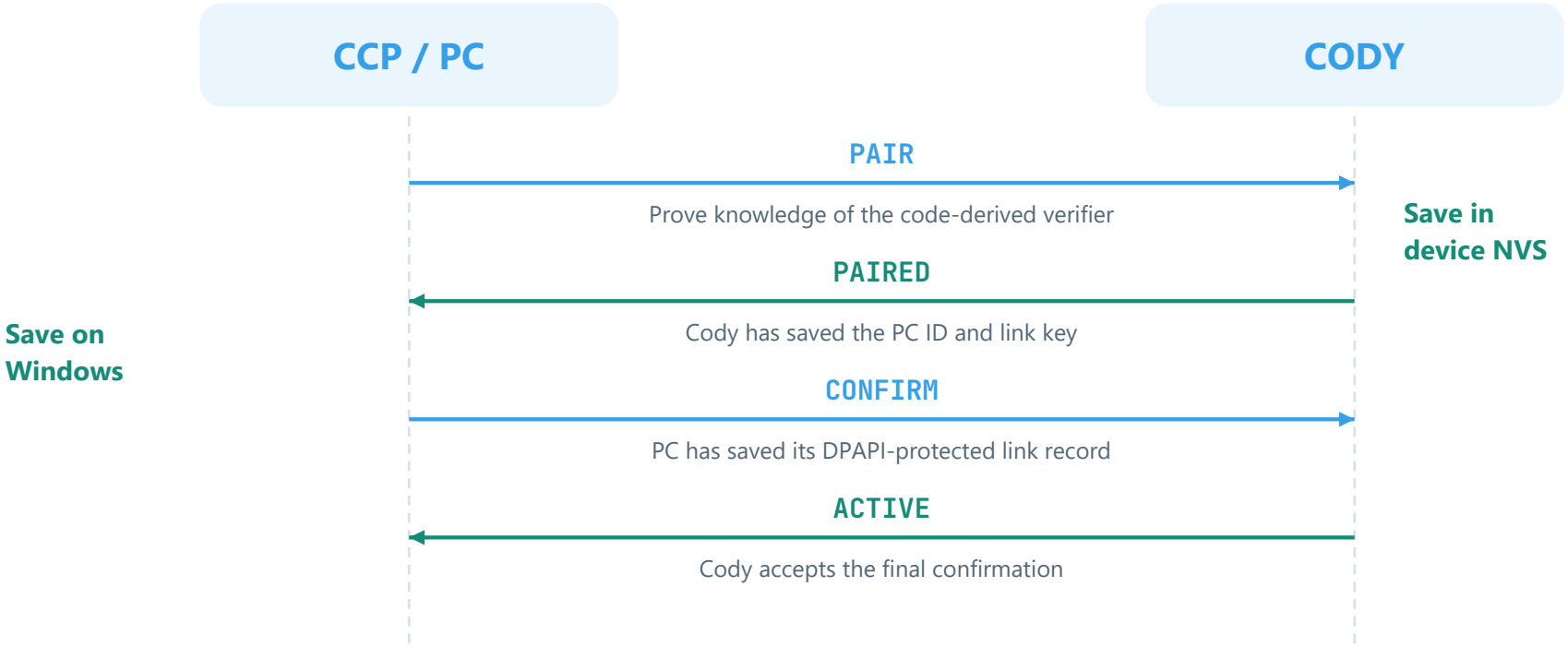
SOURCE / CCP1.md: Pairing; ccp_server.py: _matching_offers, pair

NOTES / QUESTIONS

Copyright 2026 / confidential

Pairing is complete only after confirmation

Cody and the PC each persist the relationship before the plugin reports successful pairing.



PAIR and CONFIRM retry every 0.5 s within separate 3 s windows. A short ACTIVE replay window helps recover a lost final reply. A timeout is not reported as success.

SOURCE / CCP1.md: PAIR, PAIRED, CONFIRM, ACTIVE

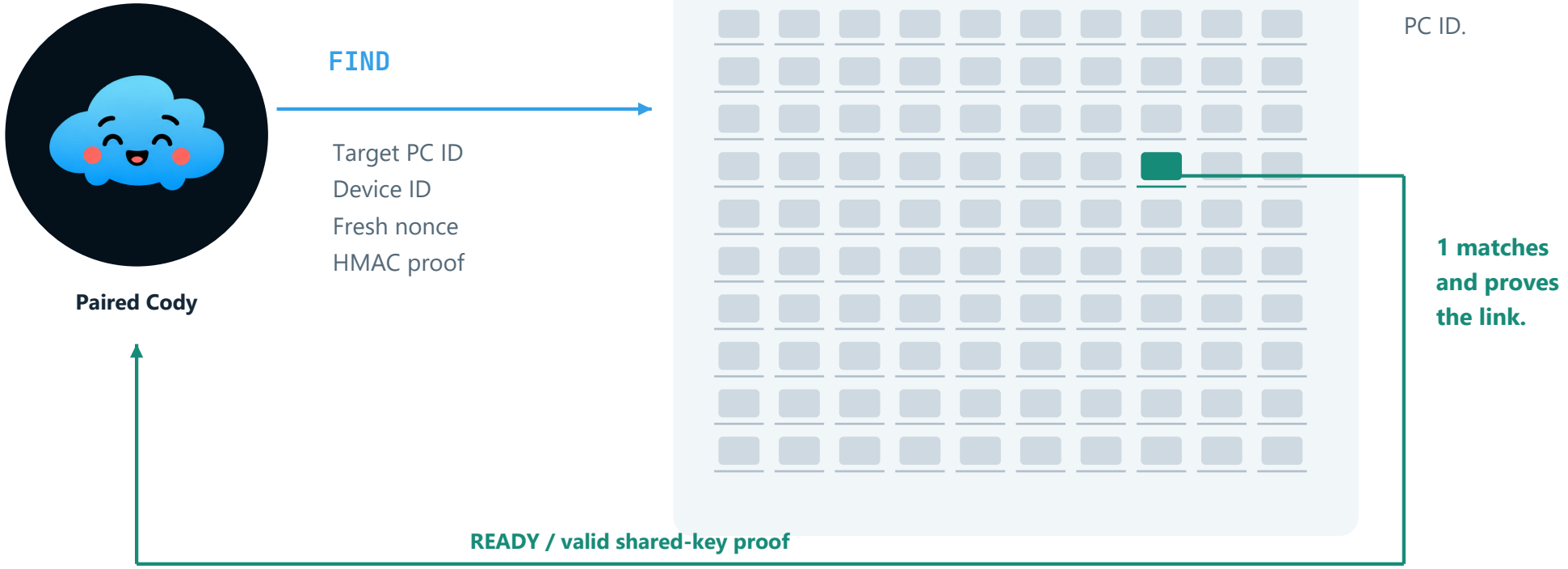
NOTES / QUESTIONS

Copyright 2026 / confidential

A hundred PCs. One retained identity.

Broadcast delivery reaches the local network; identity and proof decide who may answer.

IP addresses may change;
the paired identity stays.



Illustrative 100-PC network / not a load-test result

SOURCE / ccp_server.py: CompanionResponder._handle_find; StateStore

NOTES / QUESTIONS

Copyright 2026 / confidential

Rediscovery starts from Cody

After setup, the saved PC identity replaces the temporary display code.

13
Confirmed
Tap Let's go

→

16
Idle
10 active seconds

→

14
Find paired PC
FIND / READY

→

15
Connected
CCP is ready

Cody sends FIND
Broadcast to UDP 42118. Include the retained PC ID, device ID, fresh nonce and shared-key proof.

→

The paired CCP returns READY
Reply directly to Cody. Echo the nonce and authenticate the reply with the same retained link key.

←

Cody accepts only the expected device ID, PC ID, current nonce and proof. It learns the PC's current address from the verified reply.

SOURCE / LUNASHELL_codex_link.cpp: sendFind, acceptReady; Cody onboarding flow

NOTES / QUESTIONS

Copyright 2026 / confidential

Connected is a renewable observation

The device keeps checking; a saved pairing alone is not evidence that the PC is online.

HEALTHY LINK



NO MORE VALID REPLIES



Device view

FIND every 2 s. After 8 s without a valid READY, Cody returns to discovery.

PC status view

CCP uses a separate 10 s last-seen threshold. This is not a task/session health check.

SOURCE / LUNASHELL_codex_link.cpp: maintainState; ccp_server.py: public_status

NOTES / QUESTIONS

Copyright 2026 / confidential

Trust has a clearly defined boundary

Code-derived authentication protects selection; this is not a production security certification.

Implemented protections

A fresh pairing salt and session.
HMAC proofs bind identities and nonces.
Exactly one code-matching offer is required.
PC keys are protected by Windows DPAPI.
Cody retains the link in a versioned NVS record.

Important qualifications

The protocol is not a PAKE or TLS channel.
Captured code-derived proofs permit offline guesses; PBKDF2 increases their cost.
The NVS CRC detects record corruption; it is not encryption of the stored key.

What travels where

You enter the code into Codex, which passes it to CCP. CCP1 LAN packets contain identifiers and proofs, not the clear-text code. This is a LAN-protocol statement, not a claim about chat history or Codex data handling.

Production review still needs to cover code-guessing resistance, replay/rate limits, device key storage and hostile-network behavior.

SOURCE / CCP1.md: Security boundary; ccp_server.py: derive_verifier, StateStore

NOTES / QUESTIONS

Copyright 2026 / confidential

What must be true on a real network

The implementation is a starting point for commissioning and liveness, not an end-to-end release claim.

Network

Same reachable IPv4 broadcast domain. Private-network UDP 42117/42118 must be allowed. Guest isolation, VLAN boundaries and VPN routing can prevent discovery.

PC runtime

Windows + Python + running CCP. One CCP process owns the fixed listener per PC. A second process fails instead of sharing that socket. Not one identity per chat.

Current scope

Discover, pair and inspect liveness. No cloud discovery directory. No per-chat selection or Codex task/message transport. Multi-adapter routing needs real-network checks.

READ FROM SOURCE

Protocol messages, ports, identity checks, persistence order and configured timers.

STILL TO VERIFY END TO END

Plugin installation and lifetime in Codex; firewall and multi-adapter behavior; real Cody pairing/reconnect; packet loss, many-device load and security review. No runtime or physical tests were performed for this document.

SOURCE / README.md; CCP1.md; ccp_server.py; LUNASHELL_codex_link.cpp

NOTES / QUESTIONS

Copyright 2026 / confidential